

Distributed Sensor Hub

Final Report for the Distributed Systems Course 2025/2026

Alex Santini

`alex.santini2@studio.unibo.it`

April 24, 2026

Distributed Sensor Hub is a fully decentralized peer-to-peer platform for smart-city sensor data collection and replication, designed to operate across multiple nodes without any central coordinator. Each node may host multiple local sensors, ingest their readings through a pluggable sensor interface to support different sensor types, and maintain a local replica of the shared system state.

The platform disseminates sensor updates through push-pull gossip and resolves conflicts with deterministic timestamp-based Last-Writer-Wins merging, providing eventual consistency across replicas. In the demo setup, sensor streams are simulated for reproducibility, but the same ingestion model can be extended to real devices through dedicated adapters.

To remain operational in realistic distributed environments, the system combines gossip-based membership, phi-accrual failure detection, and anti-entropy recovery after crashes or temporary network partitions. In CAP terms, the project prioritizes availability and partition tolerance over strong consistency, while still guaranteeing convergence once communication is restored.

Fault-injection experiments on multi-node deployments show stable recovery and consistent replica alignment after failures and partitions, supporting the suitability of the approach for decentralized smart-city monitoring scenarios.

Disclaimer

During the development of the project and the preparation of this report, the author used Large Language Models (LLMs) as auxiliary tools for limited coding support, brainstorming, and improving the clarity and structure of the written text.

All outputs produced with the assistance of LLMs were critically reviewed, validated, and, when necessary, revised by the author. All architectural decisions, system design choices, and implementation details were defined and verified by the author, who takes full responsibility for the final content of this report.

1 Concept

Product type. The developed software is a distributed experimental platform composed of a set of autonomous peer-to-peer services and an optional web application for monitoring. Concretely, it combines:

- a set of autonomous peer-to-peer nodes responsible for sensor ingestion, local state management, membership handling, failure detection, and replicated state synchronization;
- an optional read-only web/introspection layer for monitoring and runtime observability.

Use case description. The software manages a cluster of sensor nodes that ingest local readings, replicate sensor state across peers, and expose runtime information for monitoring. The project is intentionally designed as a controlled distributed-systems laboratory rather than as a production-ready smart-city platform. This makes it possible to focus on decentralization, eventual consistency, membership evolution, failure detection, anti-entropy recovery, and observability under faults and network partitions.

A typical execution proceeds as follows. First, a configurable number of nodes are started. Each node may host one or more local sensors and begins ingesting readings through a pluggable sensor interface. In the current demonstration setup, sensor streams are simulated for reproducibility, but the same interface can be extended to real devices through dedicated adapters. As readings are produced, nodes disseminate updates through push-pull gossip, exchange heartbeat evidence, propagate membership information, and synchronize their replicated state. Conflicts are resolved deterministically through a *Last-Write-Wins* policy based on $(ts_ms, origin)$, so replicas progressively converge after message delays, crashes, or temporary partitions.

Typical users are smart-city monitoring stakeholders with two main human interaction profiles:

- *observational interaction* (control-room operators): inspect live sensor values, node health, and cluster evolution from monitoring workstations;

- *operational interaction* (technical operators/developers): deploy nodes, tune configuration parameters, and execute recovery or fault-handling procedures from development or administration machines.

Users are typically located wherever the system is operated or observed, such as development environments, control-room settings, or laboratory test setups. Interaction is usually periodic rather than continuous: operators interact with the system when deploying experiments, changing configuration, injecting faults, or checking recovery, while observers access the dashboard whenever live status inspection is needed. In addition, external tools may consume the exposed API to inspect, collect, or further process the data maintained by the system.

Interaction channels and devices:

- HTTP API for programmatic interaction, monitoring, and integration with external tools;
- desktop/laptop web browser for optional dashboard-based monitoring.

Data handling and roles:

- the system does not store personal user data; it stores replicated sensor-state records, timestamps, origin identifiers, membership/liveness metadata, topology views, and introspection metrics/events as node-local runtime state;
- optional log files provide operational traces for debugging and validation on the machines running the nodes;
- main roles include: *node process* (autonomous peer), *cluster operator/developer*, and *observer client* (dashboard, tests, or external tooling).

Why distribution is needed. Distribution is a core requirement, not an implementation detail:

- autonomous data collection at multiple nodes rather than at a central coordinator;
- temporary divergence of local replicas before eventual convergence;
- decentralized membership management and peer liveness assessment through heartbeat observation and phi-accrual failure detection;
- resilience under crashes, restarts, and temporary network partitions through gossip dissemination and anti-entropy recovery;
- realistic smart-city deployment assumptions, where sensing points are naturally distributed at the edge.

The system does not focus on long-term historical analytics or centralized query processing. It focuses on the information needed to study decentralized sensor-state replication: current sensor values, timestamps, origin identifiers, membership and liveness metadata, topology knowledge, and runtime observability data.

2 Requirements Elicitation and Analysis

Requirements were elicited from the intended project goal described in section 1: build a decentralized laboratory platform for distributed sensor ingestion, replicated state propagation, fault observation, and monitoring. The elicitation process therefore focused on the observable services the platform must provide to operators and observers, on the quality attributes expected from a distributed-system prototype, and on the practical constraints imposed by course evaluation and reproducible experimentation.

2.1 Glossary

- **Node:** autonomous peer process participating in the cluster and executing the full runtime stack.
- **Peer:** another node known by a node through bootstrap, heartbeat exchange, or gossip dissemination.
- **Sensor reading:** atomic observation produced by a local sensor source, including sensor identifier, value, timestamp, and origin.
- **Replicated sensor state:** node-local view of the cluster-wide latest known sensor values after merge.
- **Membership view:** node-local knowledge about known peers and their liveness status.
- **Observer:** human user or external tool consuming read-only monitoring data.
- **Operator:** human user responsible for starting nodes, configuring experiments, and injecting or observing failures.
- **Convergence:** condition in which reachable replicas expose the same winning sensor values after enough communication.
- **Partition:** temporary communication disruption that prevents some nodes from exchanging messages with the rest of the cluster.

2.2 Functional Requirements

FR1 – Multi-node autonomous operation. The system shall allow multiple nodes to start independently and participate in the same distributed deployment without requiring a central coordinator.

Acceptance criterion. When a cluster of at least two configured nodes is started, each node becomes operational as an autonomous process and exposes its local service interfaces without depending on a single master node.

FR2 – Local sensor ingestion. Each node shall be able to host one or more local sensor sources and ingest their readings into the distributed system state.

Acceptance criterion. Given at least one configured sensor on a node, the node eventually exposes fresh sensor records associated with its own origin through the read API or introspection API.

FR3 – Cluster-wide sensor-state dissemination. The system shall propagate sensor updates among nodes so that readings produced on one node become visible on other reachable nodes.

Acceptance criterion. If a node generates new sensor readings while communication paths to other nodes are available, those readings eventually appear in the replicated sensor-state view exposed by the other reachable nodes.

FR4 – Deterministic conflict resolution. The system shall resolve concurrent or conflicting updates in a deterministic way so that replicas can converge to the same winning value for each sensor record.

Acceptance criterion. When two or more updates compete for the same logical sensor record, all nodes that receive the same set of updates eventually expose the same winner for that record.

FR5 – Membership discovery and peer knowledge propagation. The system shall allow nodes to discover peers and maintain a distributed view of cluster membership.

Acceptance criterion. After startup and sufficient message exchange, a node exposes a membership view containing the peers it has discovered directly or indirectly.

FR6 – Liveness observation. The system shall provide operators with a node-local assessment of peer liveness so that missing or degraded peers can be observed during experiments.

Acceptance criterion. If a known peer becomes unreachable for long enough, at least one observing node eventually reports a degraded or failed status for that peer in its membership/introspection output.

FR7 – Recovery after crashes or temporary disconnections. The system shall allow a node that restarts or reconnects after temporary unavailability to recover the replicated state needed to rejoin normal operation.

Acceptance criterion. After a node crash or temporary isolation, once communication is restored the recovering node eventually exposes a replicated state aligned with the rest of the reachable cluster.

FR8 – Read-only observability interface. The system shall provide a read-only programmatic interface for observing sensor state, membership, topology-related information, events, and operational metrics.

Acceptance criterion. A client can query documented HTTP endpoints and obtain structured snapshots of the node's current distributed-system view without modifying runtime state.

FR9 – Human-oriented monitoring support. The system shall support human observation of the running cluster through a dashboard or equivalent visual monitoring interface.

Acceptance criterion. An observer can inspect live cluster information from a web browser by connecting to the exposed read-only API of a node.

2.3 Non-Functional Requirements

NFR1 – Decentralization. Normal operation shall not depend on any central coordinator or single control component.

Acceptance criterion. The system continues to ingest local readings, maintain local state, and serve read-only observability data on surviving nodes even if one arbitrary peer becomes unavailable.

NFR2 – Eventual consistency. The replicated sensor state shall provide eventual, not immediate, consistency across nodes.

Acceptance criterion. In the absence of new conflicting updates and after communication is restored, reachable replicas converge to the same winning values for the same sensor records.

NFR3 – Partition tolerance with degraded semantics. The system shall remain available for local operation during temporary network partitions, accepting that different partitions may temporarily diverge.

Acceptance criterion. During a partition, nodes in each connected component continue local ingestion and observation; after the partition heals, their replicas eventually converge again.

NFR4 – Fault observability. The system shall make failures and recovery visible enough to support distributed-systems experimentation and debugging.

Acceptance criterion. Operators can inspect status changes, replication activity, and recovery evidence through exposed membership, event, or metric views.

NFR5 – Reproducibility of experiments. The platform shall support repeatable validation scenarios on development machines and course evaluation environments.

Acceptance criterion. Using the documented setup, an operator can start a pre-defined topology and reproduce representative convergence and recovery experiments without relying on external managed services.

NFR6 – Extensibility of sensor sources. The system shall be open to additional sensor types without changing the conceptual role of the rest of the distributed runtime.

Acceptance criterion. New sensor sources can be introduced as additional producers while preserving the same downstream ingestion, replication, and observation capabilities.

NFR7 – Operational scalability at course-project scale. The platform shall support small and medium experimental clusters rather than only a fixed two-node demo.

Acceptance criterion. The documented deployment can be instantiated with multiple nodes and still provides replicated state and observability functions for the whole experiment.

2.4 Implementation Constraints

IR1 – Self-contained deployment. The project shall be executable in a self-contained way on commodity machines used for development and evaluation, without requiring paid or institution-managed cloud infrastructure.

Justification. This constraint follows from course-project reproducibility and portability requirements.

Acceptance criterion. The system can be started locally with the provided repository artifacts and documented dependencies.

IR2 – Reproducible multi-node setup. The project shall provide a reproducible way to instantiate representative multi-node deployments for demonstrations and validation.

Justification. The report and validation activities require repeatable distributed scenarios, including fault-injection experiments.

Acceptance criterion. An operator can launch a predefined topology from the repository and repeat the same experiment configuration with consistent system roles and interfaces.

2.5 Relevant Distributed System Features

Not all classical distributed-system features have the same priority in this project. Since *Distributed Sensor Hub* is a decentralized monitoring and experimentation platform, some features are central design drivers, while others are intentionally secondary.

Transparency. Only partially relevant. The system hides low-level transport details from observers, but it does *not* aim to hide the fact that it is distributed. On the contrary, replication lag, topology evolution, liveness suspicion, and recovery dynamics are part of what the project wants to make observable. Failure transparency is therefore limited by design.

Fault tolerance and dependability. Highly relevant. Nodes may crash, restart, or become temporarily disconnected, and the platform is expected to continue operating on surviving nodes while later recovering convergence. Availability and recoverability are more important than uninterrupted global consistency. Integrity is also relevant in the sense that replicas must converge deterministically rather than drift indefinitely.

Scalability. Relevant, but at moderate scale. The project is not meant for Internet-scale deployments; however, it must scale beyond a trivial two-node example and remain meaningful on multi-node laboratory topologies. The main concern is preserving understandable behavior as the number of peers and sensor updates grows.

Security and trust. Only marginally relevant in the current scope. The platform does not manage personal data or adversarial multi-tenant access, and the main users are trusted operators and observers in controlled environments. Security hardening is therefore not a primary project driver, although it remains a possible future extension.

Resource sharing. Relevant. Nodes share distributed knowledge rather than centralized hardware resources: replicated sensor winners, membership evidence, topology

views, and operational metadata. Coordination is required to keep those shared views convergent despite asynchrony and failures.

Openness, interoperability, and heterogeneity. Relevant. The project benefits from clean interfaces between sensor producers, node runtime, and observation clients. This matters because simulated sensors are only one current source of data, while the same ingestion boundary can later support heterogeneous adapters and external tools.

Evolvability and maintainability. Highly relevant. As a course project intended to demonstrate multiple distributed-systems concerns, the platform must remain understandable, modular, and extensible, especially for adding new sensors, observability surfaces, and validation scenarios.

Performance, concurrency, and communication efficiency. Relevant, but not under hard real-time constraints. The system executes many concurrent activities such as sensor production, replication, heartbeats, gossip exchange, and HTTP observation. Efficiency matters because excessive communication can obscure convergence behavior, but the project does not target strict latency guarantees.

Economy and costs. Relevant mainly as a practical constraint. The project should run on ordinary development hardware and avoid external infrastructure dependencies, which supports reproducibility, portability, and low evaluation cost.

3 Design

This chapter describes the high-level design adopted to satisfy the requirements in section 2. The central design goal is to support autonomous multi-node execution (FR1), cluster-wide sensor-state dissemination (FR3), deterministic convergence (FR4), recovery after failures (FR7), and strong observability (FR8–FR9), while preserving decentralization (NFR1), eventual consistency (NFR2), and partition tolerance (NFR3).

3.1 Architecture

The system follows a **fully decentralized peer-to-peer architecture** in which every node executes the same logical stack and can both produce local sensor data and consume data generated by other peers. There is no master node, no dedicated coordinator, and no centralized database. This architectural style directly addresses FR1 and NFR1: any node can participate in the cluster as an autonomous peer, and the failure of one peer does not invalidate the whole system.

From a logical viewpoint, the architecture is organized into three cooperating planes:

- a **control plane**, responsible for peer discovery, membership maintenance, liveness observation, and topology knowledge;
- a **data plane**, responsible for local sensor ingestion, local state maintenance, and state dissemination among peers;
- an **observability plane**, responsible for exposing cluster snapshots, metrics, and monitoring views to operators and observer clients.

This separation is important because the project intentionally distinguishes between *knowing who is in the system* and *replicating what the system knows about sensors*. Membership and liveness information evolve under different timing assumptions and serve different purposes than sensor-state replication. Keeping the two concerns separate improves modularity, clarity, and evolvability, which supports NFR6 and the maintainability goals discussed in section 2.5.

Within each node, the architecture is further layered:

- a **transport layer**, dealing only with network communication;
- a **protocol layer**, defining message types and dispatching received messages to the correct logic;
- a set of **domain components**, where the actual semantics of membership, failure detection, replication, topology, and observation are applied.

This layered organization keeps low-level communication concerns separate from distributed-system semantics. As a consequence, the same conceptual design would still hold if the underlying transport or encoding technology changed, which is consistent with the intended role of the design chapter.

3.2 Infrastructure

The infrastructure is intentionally minimal. The essential infrastructural unit is the **node process**, replicated one time for each participant in the cluster. Every node contains the same internal subsystems: sensor ingestion, local state management, peer membership handling, failure detection, peer-to-peer protocol handling, and a read-only HTTP observation interface. This homogeneous-node model simplifies deployment and supports both FR1 and NFR5, because the same runtime can be instantiated in different topologies without role specialization.

No external infrastructural component is required for normal operation. In particular, the design does not rely on:

- a central database;
- a message broker;
- a coordination service;
- a load balancer;
- a dedicated monitoring backend.

The only optional external-facing component is the browser dashboard, which is not part of the distributed core and behaves as a read-only observer. It can be hosted separately because it only consumes the observation API exposed by any node.

The distributed deployment model is therefore the following:

- one or more nodes run on different processes, containers, or machines;
- each node may host a different local set of sensors;
- nodes communicate directly with peers over the inter-node network;
- observers may connect to any node's read-only web interface or HTTP API.

Component discovery is bootstrapped through a configured set of initial peers. After first contact, discovery is no longer purely static: nodes can exchange peer knowledge and extend their local membership view. Naming is based on stable node identifiers plus network coordinates required for communication. This choice is sufficient for the project scope because it avoids introducing centralized service-discovery infrastructure while still allowing evolving cluster knowledge, which is coherent with FR5 and NFR1.

From a physical standpoint, the design does not assume geographic distribution at continental scale; however, it does assume that nodes may be distributed across separate machines or containers and may suffer message delay, temporary unreachability, or partial connectivity. This is exactly the failure model needed to study NFR2 and NFR3.

3.3 Modelling

The main domain entities are:

- **Node**, representing an autonomous distributed participant;
- **Peer**, representing another known node together with its membership and liveness metadata;
- **Sensor source**, representing a local producer of readings;
- **Sensor reading**, representing one observation generated by a sensor source;
- **Replicated sensor record**, representing the current winning value known for one logical sensor identifier;
- **Membership entry**, representing a node's local belief about another peer's status;
- **Topology entry**, representing adjacency knowledge useful for network observability and connection policies;
- **Observer view**, representing an aggregated, read-only projection of internal distributed state.

These entities map naturally to infrastructural components. Sensor sources are local to a single node, while replicated sensor records, membership entries, and topology entries are stored as node-local replicas. In other words, the design does not centralize domain state in one authoritative repository; instead, each node stores its own best-effort replica of the distributed knowledge relevant to operation and observation.

The most important domain events are:

- production of a local sensor reading;
- discovery of a new peer;
- reception of a heartbeat;
- transition of a peer between liveness states;
- emission or reception of a sensor-state update;
- request for missing replication data;
- full-state transfer during recovery;
- generation of an introspection snapshot for observers.

The exchanged messages can be grouped into three categories:

- **control messages**, for discovery, membership dissemination, and liveness observation;

- **state-replication messages**, for sensor update propagation and catch-up;
- **read-only queries**, for HTTP observation by users or tools.

The system state comprehends:

- latest winning sensor values together with origin and timestamp metadata;
- membership knowledge about known peers;
- liveness evidence and peer-status information;
- topology knowledge and connection-related views;
- operational events and metrics used for observability.

This model is intentionally centered on *current replicated knowledge*, not on long-term historical storage. That design keeps the project focused on convergence, liveness, and fault handling rather than on archival analytics.

3.4 Interaction

There are two main interaction families in the system.

The first is **peer-to-peer interaction among nodes**. This interaction is asynchronous and message-based. It is used for:

- bootstrap and peer discovery;
- heartbeat exchange;
- dissemination of membership knowledge;
- dissemination and retrieval of replicated sensor state;
- recovery synchronization after missed updates.

The second is **observer interaction**, where external clients query a node through the HTTP read API. This interaction is request-response and read-only. It does not participate in cluster coordination and it never mutates the distributed state.

At the inter-node level, communication follows a layered path:

$$\begin{aligned} & domain\ intent \rightarrow protocol\ message \rightarrow transport\ delivery \rightarrow protocol\ dispatch \\ & \rightarrow domain\ update \end{aligned}$$

The main interaction patterns are:

- **bootstrap request/response**, used when a node joins and needs initial peer knowledge;
- **periodic heartbeat exchange**, used to collect liveness evidence;

- **gossip dissemination**, used to spread membership knowledge without central coordination;
- **push-pull state synchronization**, used to exchange recent sensor-state updates efficiently;
- **full synchronization on demand**, used when incremental recovery is insufficient;
- **polling-based observation**, used by the dashboard and external tools to inspect cluster state.

This design supports FR3, FR5, FR6, FR7, and FR8 because it combines low-coupling peer communication with an observation interface that stays orthogonal to the replicated core.

3.5 Behaviour

The behavior of the main components can be summarized as follows.

Sensor sources are active components: they periodically generate local readings and inject them into the node. They are conceptually producers and do not need global knowledge.

The local state manager is stateful and authoritative for the node's replica. It receives local and remote updates, applies deterministic merge rules, stores the current winners, and exposes snapshots to both replication and observability components. This component is central to FR2, FR3, and FR4.

The membership manager is also stateful. It stores known peers, updates direct or indirect liveness evidence, and maintains the node's local view of cluster membership.

The failure detector behaves as a local evaluator rather than as a replicated authority. It interprets heartbeat timing and produces suspicion levels for peers observed directly by the node. Those local judgements are then translated into membership information that can be disseminated.

The heartbeat and gossip runtime is periodic. At each round, it evaluates liveness, sends heartbeat probes, and disseminates updated membership knowledge. Its purpose is not to guarantee exact membership synchronization at every instant, but to make peer-status knowledge progressively converge.

The replication runtime is periodic as well. It pushes recent sensor-state updates to selected peers and pulls missing updates from others. When incremental recovery is not sufficient, it can trigger a broader synchronization phase. This behavior is essential for FR7 and NFR2.

The observability subsystem is read-only. It collects snapshots from state, membership, topology, and event/metric providers, then publishes a coherent observation view for users and tools.

State updates are therefore concentrated in a small set of stateful components:

- local sensor-state storage;

- local membership storage;
- local topology and observability stores.

Other components mainly trigger transitions or move data between these stores. This separation keeps write paths explicit and supports maintainability.

3.6 Data and Consistency Issues

The system stores several forms of node-local runtime data:

- the replicated sensor-state winners;
- the incremental replication view needed for propagation and recovery;
- the membership table and liveness metadata;
- topology knowledge;
- recent events and metrics for introspection.

This data is primarily **runtime state**, not long-term persistent storage. The design does not require an external database because the goal is to study distributed replication and convergence of current system knowledge rather than durable business records. This keeps the architecture lightweight and self-contained, matching IR1.

Conceptually, the replicated sensor state behaves like a key-value map from a logical sensor identifier to the current winning record. The important property is not relational querying power, but deterministic merge semantics. For this reason, the core consistency problem is solved at the application level rather than delegated to a storage engine.

Consistency is handled differently for different data types:

- **sensor state** uses eventual consistency with deterministic Last-Writer-Wins conflict resolution based on timestamp and origin metadata;
- **membership state** also converges eventually, but it reflects local observations that may temporarily differ across nodes;
- **observability data** is inherently approximate and time-sensitive, because it represents a snapshot of changing distributed state.

This means the system explicitly rejects strong consistency as a primary goal. During partitions or transient delays, different nodes may expose different but still valid local views. Once communication resumes and no newer conflicting updates appear, the replicas converge. This choice is aligned with NFR2 and NFR3.

Data is shared among components in well-defined ways:

- sensor producers share generated readings with the local state manager;
- the state manager shares snapshots and deltas with the replication runtime and the observability subsystem;

- the membership subsystem shares peer-status information with heartbeat, gossip, topology, and observability components;
- the observability subsystem reads from other stores but does not modify them.

This read/write asymmetry is deliberate: observation must remain decoupled from mutation so that FR8 can be satisfied without introducing accidental feedback paths.

3.7 Fault-Tolerance

Fault tolerance is one of the central design drivers of the project. The first mechanism is **state replication**: sensor-state knowledge is disseminated among peers so that the system does not depend on a unique owner of cluster knowledge. Replication is best-effort and decentralized, which supports NFR1 and NFR3 even though it does not provide instantaneous consistency.

The second mechanism is **heartbeat-based liveness observation**. Nodes periodically exchange liveness probes and evaluate the timing of received heartbeats. This allows each node to form a local judgement about whether a peer is alive, suspected, or dead. Since such knowledge is local and timing-dependent, the design treats liveness as a continuously revised estimate rather than as an absolute truth.

The third mechanism is **membership gossip**. A node does not keep liveness knowledge only for itself; it disseminates updated membership information so that peer knowledge can propagate even when direct connectivity patterns are incomplete. This improves resilience of membership awareness and helps the system maintain a broader view of the cluster.

The fourth mechanism is **anti-entropy recovery**. Incremental propagation is efficient when peers stay mostly synchronized, but it may be insufficient after outages, restart, or missed traffic. For this reason, the design includes a stronger recovery path that allows a lagging node to regain a coherent replicated view after disruption, directly supporting FR7.

Error handling is intentionally failure-aware rather than failure-oblivious. If a component or peer becomes temporarily unavailable:

- surviving nodes continue local operation;
- local replicas may diverge temporarily;
- membership views reflect degraded reachability;
- synchronization resumes when communication becomes possible again.

This behavior is preferable to halting the whole system because the project prioritizes availability and observation under faults over strong global coordination.

3.8 Availability

The design does not introduce classical web-style caching layers or external load balancers, because the system is not centered on high-volume client traffic toward a centralized service. Instead, availability is achieved structurally through replication, autonomy of nodes, and local read access to each node’s current view.

Each node acts as a locally available observation point. As long as a node remains running, it can still:

- ingest its own local sensor readings;
- maintain its local replica;
- expose read-only snapshots to operators and tools.

In this sense, replicated state itself plays a role analogous to availability-oriented caching: each node keeps a local working copy of shared cluster knowledge, so read operations do not require contacting a central authority.

No explicit load balancing is required because there is no single mandatory ingress point for observation. Different observers may query different nodes, and each node exposes its own local view. This is sufficient for the experimental scope of the project.

During a **network partition**, the design favors continued local operation over blocking. Nodes inside each connected component continue to exchange information among themselves, while nodes separated by the partition stop receiving fresh updates from one another. Consequently:

- sensor-state replicas may diverge temporarily across partitions;
- membership views may mark remote peers as suspected or dead;
- local observation remains available inside each connected component.

When the partition heals, incremental dissemination and recovery mechanisms progressively re-align the replicas. This behavior is a direct consequence of choosing availability and partition tolerance over strong consistency, and it is exactly the semantics required by NFR2 and NFR3.

3.9 Security

Security is intentionally limited in the current design because, as discussed in section 2.5, it is not a primary driver of this project. The system is assumed to run in controlled laboratory, development, or trusted demonstration environments.

Accordingly, the current design does **not** make authentication a core architectural dependency for either peer-to-peer traffic or read-only observation endpoints. Similarly, it does not define a rich authorization model with differentiated user permissions. The effective roles are simple: nodes participate in the distributed protocol, while observers query read-only information.

This choice keeps the architecture aligned with the educational scope of the project and avoids obscuring the main distributed-systems concerns with security infrastructure that is orthogonal to the report goals. However, the design has been kept modular enough that stronger security policies could be introduced later at clear boundaries:

- on node-to-node channels, to authenticate peers and protect inter-node communication;
- on HTTP observation endpoints, to restrict access to monitoring data;
- on deployment boundaries, through network isolation and reverse-proxy policies.

Likewise, no cryptographic protocol is a conceptual prerequisite of the current system model. Confidentiality and strong identity guarantees are therefore outside the present scope and remain future extensions rather than core design commitments.

4 Implementation

This chapter reports the concrete technology-dependent choices used to realize the design discussed in section 3. The implementation deliberately favors a lightweight and inspectable stack, so that the distributed behavior of the system remains visible and reproducible without relying on heavy external infrastructure.

4.1 Protocols and Data Representation

Two communication protocols are used, each chosen according to the interaction style required by the system.

Node-to-node communication. Peer communication is implemented over **TCP**. This choice was preferred over UDP because the project needs ordered and reliable delivery of protocol payloads at the transport level, while still keeping the communication stack simple and under direct control. TCP is therefore used for:

- bootstrap and peer discovery traffic;
- heartbeat exchange;
- membership gossip dissemination;
- sensor-state update propagation;
- incremental and full synchronization during recovery.

Rather than adopting an external RPC or messaging framework, the project uses a custom lightweight application protocol on top of TCP. Payloads are transmitted as length-prefixed frames, which makes message boundaries explicit and keeps the implementation compatible with a persistent stream-oriented transport.

Observer communication. Human users and external tools interact with the system through a **read-only HTTP API**. HTTP is appropriate here because the observability plane is request-response oriented, browser-friendly, and naturally suited to dashboards, integration scripts, and test harnesses. Since the observation API does not participate in peer coordination, it is intentionally separated from the internal peer-to-peer protocol.

In-transit representation. Messages and HTTP responses are represented as **JSON**. This choice was motivated by readability, ease of debugging, and direct interoperability with browsers and Python tooling. JSON is sufficient for the project because payload sizes are moderate and the main goal is clarity rather than binary compactness. Message validation and dispatch are then handled at the protocol layer after decoding.

The main message families implemented in the peer protocol are:

- `JOIN_REQUEST` and `PEER_LIST` for discovery;
- `PING` and `PONG` for heartbeat-based liveness observation;
- `GOSSIP_STATE` for disseminating membership knowledge;

- `SENSOR_UPDATE` for push-based sensor replication;
- `GET_DELTA` and `DELTA_UNAVAILABLE` for incremental recovery;
- `FULL_SYNC_REQUEST` and `FULL_SYNC_RESPONSE` for stronger catch-up after disruption.

4.2 State Management and Storage Choices

The implementation does **not** use an external database. All relevant information is maintained as in-memory runtime state inside each node:

- the current winning sensor records;
- replication deltas and sequence metadata;
- the peer membership table;
- liveness-related information;
- topology snapshots;
- introspection events and counters.

This is consistent with the project scope: the goal is to study replication, convergence, recovery, and observability of current distributed state, not durable long-term business storage. As a consequence, no SQL or NoSQL query layer is required. Queries are replaced by direct access to in-memory state providers, which are then exposed through the HTTP observation endpoints.

From an implementation viewpoint, the most important storage choice is therefore not the selection of a database technology, but the decision to encode merge semantics explicitly in the application logic. Sensor-state convergence is implemented with deterministic Last-Writer-Wins conflict resolution based on timestamp and origin metadata, while membership and liveness views are maintained through dedicated in-memory structures updated by protocol handlers and periodic runtimes.

4.3 Authentication and Authorization

No explicit authentication or authorization protocol is implemented in the current version of the system. This is a deliberate scope decision, not an omission by accident. The project runs in controlled development and laboratory environments, and the report prioritizes decentralization, consistency, failure handling, and observability over access-control concerns.

Accordingly:

- peer-to-peer channels do not currently require node authentication;
- the HTTP API does not require login, tokens, or session management;

- the system does not implement role-based or attribute-based authorization.

The practical effect is that any process able to reach a node in the current deployment can interact with the exposed interface allowed by that channel. This is acceptable for the project scope, but it also marks a clear limitation of the current implementation. Stronger authentication, endpoint protection, and transport hardening are left as future work rather than embedded in the present runtime.

4.4 Technological details

Programming language and runtime. The project is implemented in **Python**. This choice supports rapid experimentation, readable code, and low setup cost for a course project. The implementation makes extensive use of the Python standard library for networking, concurrency, HTTP serving, serialization, logging, and timing, which keeps external dependencies intentionally minimal.

Concurrency model. The runtime is based on **threads**, background worker loops, and in-memory queues. This model is used for several long-lived activities that must proceed concurrently:

- accepting inbound TCP connections;
- maintaining outbound peer workers;
- periodic heartbeat and failure-detector evaluation;
- periodic push-pull replication;
- sensor generation;
- HTTP serving for observation.

This approach was preferred to a more complex actor framework or asynchronous event-loop architecture because it keeps the execution model explicit and understandable, which is desirable for an educational distributed-systems project.

Configuration management. Node configuration is externalized through **environment variables**. Parameters such as node identity, ports, bootstrap peers, sensor definitions, replication cadence, failure-detector thresholds, and fault-injection options are all configurable from the environment. This makes it easy to instantiate different topologies and experimental scenarios without changing source code.

Sensor implementation. Sensor sources are implemented as pluggable local producers. The repository includes multiple simulated sensor types, such as numeric, boolean, wave, noise, spike, trend, categorical, incremental, and finite test sensors. This confirms the extensibility objective identified in NFR6 while keeping the whole platform reproducible in the absence of real hardware devices.

Networking layer. The transport layer is implemented with Python sockets and a custom framed TCP protocol. On the outbound side, each peer connection has its own worker and FIFO send queue; on the inbound side, a threaded TCP server accepts

connections, reads framed messages, validates them, and dispatches them to protocol handlers. The implementation also includes bounded worker counts and configurable socket limits, which helps prevent unbounded resource growth in larger experiments.

HTTP observation layer. The Web API is implemented with Python’s ‘http.server’ facilities, in particular a threaded HTTP server and custom request handlers. This is sufficient because the API is read-only and relatively small, so introducing a larger web framework would not provide proportionate benefits for the project scope.

Containerization and deployment automation. The project uses **Docker** and **Docker Compose** to instantiate reproducible node topologies. Multiple compose configurations are provided, including small and medium cluster layouts, so that the same codebase can be exercised under different scales and fault scenarios. Containerization strongly supports NFR5 because it reduces environmental variability across test runs and demonstrations.

Testing and quality tools. Validation and development are supported by:

- **pytest** for unit and integration testing;
- Docker-backed integration harnesses for multi-node and fault-recovery scenarios;
- **pyright** for static type checking;
- structured logging and introspection endpoints for runtime diagnosis.

Dependencies. External dependencies are intentionally few. Besides development tools, the main runtime dependency is ‘python-dotenv’, used to simplify environment-based configuration. This minimal dependency footprint is coherent with the project’s emphasis on portability, inspectability, and self-contained deployment.

5 Validation

La validazione del progetto è stata progettata su più livelli, in modo da verificare sia la correttezza funzionale dei singoli moduli sia il comportamento emergente del sistema distribuito in presenza di replicazione, guasti e riconnessione. La strategia adottata combina test unitari, test di integrazione, smoke test in ambiente containerizzato e scenari manuali di osservazione, con l'obiettivo di coprire l'intero ciclo di vita di un nodo: avvio, bootstrap, propagazione dello stato, failure detection, recovery e introspection.

5.1 Automatic Testing

Test unitari. La base del piano di validazione è costituita da una suite di test unitari sviluppata con `pytest`. I test unitari verificano in isolamento le responsabilità dei moduli principali del sistema:

- `tests/protocol/`: correttezza di serializzazione, deserializzazione, validazione e dispatch dei messaggi del protocollo applicativo;
- `tests/networking/`: framing TCP, gestione delle connessioni client/server, limiti operativi e robustezza del trasporto;
- `tests/state/`: applicazione deterministica della politica Last-Writer-Wins e mantenimento dei delta incrementali;
- `tests/membership/` e `tests/fd/`: aggiornamento della vista membership, heart-beat handling e stima *phi-accrual* per la classificazione dei peer;
- `tests/sensors/`: comportamento dei generatori di sensori simulati e del relativo lifecycle manager;
- `tests/webapi/` e `tests/introspection/`: coerenza degli endpoint HTTP in sola lettura e del contratto di introspection esportato ai client.

Il razionale di questa suite è duplice: da un lato impedire regressioni nei componenti a semantica locale, dall'altro rendere esplicite le invarianti che il sistema deve preservare indipendentemente dalla topologia distribuita. Al momento della verifica locale del repository, l'esecuzione del comando `python -m pytest -m "not integration" -q` ha prodotto 180 `passed`, 9 `deselected`, confermando la stabilità della parte non integrata della suite.

Test di integrazione in-process. Una seconda fascia di validazione esercita l'interazione tra più nodi reali eseguiti nello stesso processo di test o in processi coordinati da harness dedicati. In particolare:

- `tests/integration/test_deterministic_state_convergence.py` verifica la convergenza dello stato replicato in un cluster di sei nodi, controllando che i vincitori LWW finali coincidano con quelli attesi;

- `tests/integration/test_cluster_harness.py` valida il corretto funzionamento dell'infrastruttura di supporto per scenari distribuiti riproducibili;
- `tests/integration/test_finite_test_sensor.py` usa sorgenti deterministiche per rendere osservabili e confrontabili le sequenze di aggiornamenti tra nodi.

Questi test coprono proprietà che non possono essere inferite dalla sola correttezza locale: propagazione incrementale, ordine relativo degli aggiornamenti, bootstrap iniziale e convergenza eventuale dopo scambi multipli di messaggi.

Test di integrazione Docker-based e scenari production-like. La terza fascia di validazione impiega topologie Docker Compose per esercitare il software in un ambiente più vicino al deployment reale. In questa categoria rientrano:

- `tests/integration/test_docker_deterministic_state_convergence.py`, che replica la verifica di convergenza su nodi containerizzati;
- `tests/integration/test_docker_crash_restart_recovery.py`, che valuta il recupero dopo arresto e riavvio di un nodo;
- `tests/integration/test_docker_partition_reconciliation.py`, che verifica la riconciliazione dello stato dopo una discontinuità di connettività;
- `tests/integration/verify_cluster.py`, usato come smoke test per controllare l'avvio corretto del cluster e la raggiungibilità degli endpoint principali.

Il vantaggio di questa soluzione è che lo stesso assetto di deployment impiegato per la dimostrazione viene riutilizzato come infrastruttura di test. In questo modo si riduce la distanza tra ambiente di sviluppo, ambiente di verifica e ambiente di presentazione, migliorando la riproducibilità degli esperimenti.

Automazione dell'esecuzione. L'esecuzione automatica dei test è supportata anche da workflow CI definiti in `.github/workflows/unit-tests.yml` e `.github/workflows/integration-test`. La pipeline di unit test installa le dipendenze e lancia `pytest` escludendo i test di integrazione, mentre la pipeline di integrazione esegue sia un controllo di convergenza deterministica sia uno smoke test su cluster Docker. Questa automazione permette di intercettare regressioni sia nella logica locale sia negli scenari distribuiti più significativi.

Come eseguire i test. Le modalità principali di esecuzione, già documentate nel repository, sono le seguenti:

- test unitari e modulari: `pytest --maxfail=1` oppure `python -m pytest -v -m "not integration"`;
- test di protocollo: `pytest -m protocol`;
- verifica mirata della convergenza LWW: `python -m pytest tests/integration/test_determinist -q -s`;
- smoke test Docker: avvio della topologia Compose seguito da `python tests/integration/verify_cl --timeout 60 --interval 2`.

Copertura rispetto agli obiettivi progettuali. Nel loro insieme, i test automatizzati coprono i punti più importanti emersi nei requisiti di progetto:

- correttezza del protocollo di comunicazione e delle sue validazioni;
- convergenza deterministica dello stato replicato;
- propagazione della membership e dell'informazione di liveness;
- tolleranza a crash, riavvio e partizione temporanea;
- verificabilità esterna tramite API HTTP e introspection.

5.2 Acceptance test

Accanto alla validazione automatica sono stati eseguiti anche test manuali di accettazione, orientati soprattutto agli aspetti osservabili e sperimentali del sistema. Tali prove sono state condotte mediante:

- avvio di cluster Docker a 2, 6 e 12 nodi;
- consultazione diretta degli endpoint `/api/state`, `/api/membership`, `/api/introspection` e delle relative viste derivate;
- utilizzo della dashboard statica in `web/` per osservare topologia, eventi recenti, metriche di replica e stato dei peer;
- esecuzione dello script `manual_tests/compose_chaos.py` per arrestare e riavviare selettivamente un servizio durante l'esecuzione del cluster.

Questi test manuali hanno avuto l'obiettivo di confermare proprietà difficili da catturare completamente con sole asserzioni automatiche: leggibilità dello stato esportato, qualità dell'osservabilità, coerenza temporale delle dashboard e comportamento percepibile del sistema durante riconfigurazioni e transienti di rete.

La componente manuale non sostituisce la suite automatica, ma la integra. In particolare, è risultata utile quando il criterio di accettazione era anche qualitativo, ad esempio nella valutazione della chiarezza delle informazioni mostrate all'utente o nella diagnosi visuale di fenomeni come pull storm, ritardi accumulati o rapidi cambi di stato nella membership.

6 Deployment

Il deployment del progetto è stato progettato con due obiettivi complementari: permettere l'esecuzione rapida su una singola macchina per attività di sviluppo e debug, e garantire al tempo stesso la riproducibilità di topologie multi-nodo tramite containerizzazione. La distribuzione del sistema non richiede infrastrutture esterne, orchestratori dedicati o servizi persistenti di supporto: ogni nodo esegue in autonomia la propria logica di rete, replica, failure detection e osservabilità HTTP.

6.1 Prerequisiti software

Per eseguire il progetto da zero sono necessari i seguenti strumenti:

- una versione recente di **Python 3**; la pipeline CI del repository valida esplicitamente l'esecuzione con `Python 3.12`;
- **pip** per l'installazione delle dipendenze Python;
- **Docker** e **Docker Compose** per l'avvio delle topologie containerizzate;
- facoltativamente, un browser Web per consultare la dashboard di osservabilità e gli endpoint HTTP esposti dai nodi.

Le dipendenze runtime sono volutamente minime e sono definite in `requirements.txt`; gli strumenti di sviluppo e analisi statica sono invece raccolti in `requirements-dev.txt` e nel file `package.json` per la sola dipendenza di `pyright`.

6.2 Installazione locale

L'installazione base del progetto segue una procedura lineare:

1. clonare il repository;
2. entrare nella directory del progetto;
3. installare le dipendenze Python richieste.

Una possibile sequenza operativa è la seguente:

```
git clone https://github.com/AlexSantini10/distributed-sensor-hub.git
cd distributed-sensor-hub
pip install -r requirements.txt
```

Al termine dell'installazione, il repository è pronto sia per l'esecuzione diretta di un singolo nodo sia per il lancio dei test automatici. Un primo controllo consigliato consiste nell'eseguire `python -m pytest -m "not integration" -q`, così da verificare immediatamente la coerenza dell'ambiente locale.

6.3 Configurazione tramite environment variables

L'intera configurazione operativa del nodo è esternalizzata attraverso variabili d'ambiente. Questo approccio rende il deployment semplice da replicare sia in locale sia nei container, evitando la dipendenza da file di configurazione rigidi e consentendo di descrivere topologie diverse senza modificare il codice.

Il file `.env.example` documenta le variabili principali, tra cui:

- identificazione e binding del nodo: `NODE_ID`, `HOST`, `PORT`, `WEB_API_PORT`;
- bootstrap iniziale: `BOOTSTRAP_PEERS`;
- limiti del server TCP: `SOCKET_TIMEOUT`, `ACCEPT_QUEUE_SIZE`, `MAX_CONNECTIONS`, `MAX_WORKERS`;
- parametri del failure detector: `PHI_THRESHOLD_SUSPECT`, `PHI_THRESHOLD_DEAD`, `PHI_INITIAL_INTERVAL`;
- cadenza della replica push/pull: `GOSSIP_SYNC_INTERVAL_MS`, `GOSSIP_PUSH_*`, `GOSSIP_PULL_*`;
- simulazione di ritardi e perdite di rete: `NETWORK_DELAY_*`, `NETWORK_PACKET_LOSS_PROB`;
- definizione dei sensori locali: `SENSORS` e famiglia `SENSOR_<i>_*`.

Il risultato pratico è che la stessa immagine software può assumere ruoli diversi semplicemente cambiando il set di variabili, caratteristica utile sia per i test sia per le dimostrazioni in aula.

6.4 Esecuzione di un singolo nodo

Per un'esecuzione minimale in locale è sufficiente definire alcune variabili essenziali e avviare `node.py`. Un esempio in PowerShell è:

```
$env:NODE_ID="node-1"
$env:HOST="0.0.0.0"
$env:PORT="9000"
$env:BOOTSTRAP_PEERS=""
$env:WEB_API_PORT="10000"
python node.py
```

L'esito atteso è l'avvio del nodo con listener TCP e server HTTP attivi. In questa modalità il sistema è già interrogabile tramite API Web, ad esempio sugli endpoint `/api/state`, `/api/membership` e `/api/introspection`.

6.5 Deployment containerizzato

Il deployment multi-nodo è basato su Docker Compose. Il repository include più file Compose, ciascuno orientato a un diverso scenario sperimentale:

- `docker/docker-compose-base.yml`: topologia minima e veloce da avviare, utile per smoke test e demo rapide;
- `docker/docker-compose-6-nodes.yml`: scenario intermedio, adatto a osservare in modo chiaro la convergenza dello stato e i fenomeni di gossip;
- `docker/docker-compose-12-nodes.yml`: scenario più denso, pensato per stressare la disseminazione e la leggibilità dell'osservabilità;
- file Compose dedicati ai test di integrazione: usati per scenari deterministici e validazione automatica.

L'avvio avviene con comandi del tipo:

```
docker compose -f docker/docker-compose-base.yml up --build -d
docker compose -f docker/docker-compose-6-nodes.yml up --build -d
docker compose -f docker/docker-compose-12-nodes.yml up --build -d
```

Il risultato atteso è la creazione di più container omogenei, ciascuno con:

- una propria identità logica (`NODE_ID`);
- porte TCP e HTTP distinte esposte verso l'host;
- set dedicati di sensori simulati;
- parametri di rete e replica configurati in modo indipendente;
- cartella log condivisa montata come volume per l'ispezione esterna.

6.6 Ingegnerizzazione dei file Compose

I file `docker-compose` non si limitano a istanziare copie identiche del nodo, ma codificano scenari di laboratorio con profili eterogenei. In particolare:

- i peer di bootstrap sono distribuiti tra i nodi per evitare una dipendenza da un singolo punto iniziale di contatto;
- i sensori differiscono per tipo, periodicità, latenza e unità di misura, così da simulare carichi non uniformi;
- alcuni nodi introducono deliberatamente ritardi, jitter o probabilità di perdita più elevate, in modo da rendere osservabili fenomeni di degrado e recovery;
- la dimensione del buffer dei delta replicati e le soglie di push/pull sono esplicitate nel deployment, facilitando la taratura sperimentale.

Questa scelta è rilevante dal punto di vista metodologico: il deployment è parte integrante dell'esperimento distribuito, non un semplice dettaglio operativo. I file Compose fungono quindi sia da strumenti di esecuzione sia da descrizioni riproducibili degli scenari analizzati durante la validazione.

6.7 Verifica del corretto avvio

Dopo il deployment, la verifica può essere condotta a tre livelli:

- livello infrastrutturale, controllando lo stato dei container con `docker compose ps`;
- livello applicativo, interrogando gli endpoint HTTP di uno o più nodi;
- livello sistemico, eseguendo `python tests/integration/verify_cluster.py` per confermare startup, raggiungibilità e disponibilità di base del cluster.

In questo modo il deployment non viene considerato completo solo quando i processi risultano attivi, ma quando il cluster esibisce effettivamente il comportamento distribuito atteso.

6.8 Continuous delivery dell'immagine Docker

Per semplificare il riuso del nodo al di fuori dell'ambiente di sviluppo locale, il repository include anche una pipeline di continuous delivery basata su GitHub Actions. A ogni push sul branch `main`, il workflow costruisce l'immagine del nodo a partire da `docker/Dockerfile.base` e la pubblica su GitHub Container Registry, mantenendo aggiornati i tag "rolling" come `latest` e `main`.

La creazione di una GitHub Release avviene invece solo quando a un commit viene associato un tag di versione, ad esempio `v1.0.0`. In quel caso il workflow pubblica anche il tag versionato dell'immagine e genera una release dedicata, contenente il digest dell'artefatto e un piccolo bundle operativo: istruzioni sintetiche, file `.env` di esempio e un file `docker-compose` minimale già vincolato all'immagine rilasciata. Questo separa il flusso di integrazione continua dalla pubblicazione di versioni esplicite destinate alla distribuzione.

Questa scelta migliora la tracciabilità del deployment: il legame tra sorgente versionato, immagine costruita e istruzioni operative rimane esplicito, facilitando sia le dimostrazioni sia l'eventuale riproduzione di uno specifico stato del sistema distribuito.

7 User Guide

Questa sezione descrive l'uso operativo del sistema dal punto di vista dell'utente tecnico che desidera avviare il cluster, osservare la convergenza dello stato e consultare le superfici di introspection. Poiché il progetto non espone funzionalità di scrittura verso l'esterno, l'interazione principale consiste nell'avviare i nodi, attendere l'evoluzione del sistema distribuito e analizzarne il comportamento mediante API HTTP e dashboard Web.

7.1 Avvio del sistema

L'utilizzo più efficace per la dimostrazione consiste nell'avviare una topologia Docker già predisposta. Lo scenario consigliato è quello a sei nodi:

```
docker compose -f docker/docker-compose-6-nodes.yml up --build -d
```

Dopo l'avvio è opportuno attendere alcuni secondi affinché i nodi completino bootstrap, scambio della peer list, primi heartbeat e prime repliche di stato. L'effetto atteso è la progressiva comparsa di una vista condivisa della membership e di record sensore replicati fra i nodi.

Per arrestare il cluster:

```
docker compose -f docker/docker-compose-6-nodes.yml down
```

7.2 Consultazione diretta delle API

Ogni nodo espone una Web API read-only sulla porta definita da `WEB_API_PORT`. Gli endpoint più utili durante l'uso sono:

- `/api/state`: snapshot dello stato sensori replicato;
- `/api/updates`: stream o vista degli aggiornamenti recenti;
- `/api/membership`: vista locale della membership;
- `/api/topology`: rappresentazione della topologia conosciuta;
- `/api/introspection`: snapshot aggregato secondo il contratto `introspection/v1`;
- `/api/introspection/events`: eventi recenti di control plane;
- `/api/introspection/metrics`: contatori e indicatori relativi alla replica.

Ad esempio, in una topologia base, il nodo `node-1` è normalmente raggiungibile su `http://localhost:10000`. Interrogando l'endpoint aggregato `/api/introspection` si ottiene una vista utile a verificare:

- quali peer sono attualmente considerati `alive`, `suspected` o `dead`;
- quali record sensore sono noti al nodo interrogato;

- quanti round di replica e messaggi di gossip sono stati osservati;
- se il buffer dei delta è sufficiente oppure produce `DELTA_UNAVAILABLE`.

7.3 Uso della dashboard Web

Per un'osservazione più comoda è disponibile una dashboard statica nella directory `web/`. Essa non richiede backend aggiuntivi e consuma esclusivamente l'endpoint `/api/introspection`. Per avviarla:

```
cd web
python -m http.server 8080
```

Successivamente è sufficiente aprire nel browser l'indirizzo `http://localhost:8080` e impostare come *API Base URL* l'endpoint HTTP di uno dei nodi, ad esempio `http://localhost:10000`. La dashboard rende disponibili le principali viste di analisi:

- grafo della topologia osservata;
- stato dei peer e relativa semantica di liveness;
- tabella dei sensori replicati con origine e timestamp;
- timeline degli eventi recenti;
- striscia di metriche sintetiche per gossip, full sync e delta replication.

L'esito atteso, in un cluster stabile, è una topologia connettivamente coerente, una majority dei peer in stato `alive` e timestamp dei sensori relativamente recenti per le sorgenti a frequenza elevata.

7.4 Interpretazione dei principali segnali osservabili

Durante l'uso del sistema è utile interpretare alcuni indicatori operativi:

- se compaiono molti eventi `sensor_update_received` con `applied:false`, significa che il nodo riceve aggiornamenti già superati dal criterio LWW;
- se il contatore `get_delta_unavailable_total` cresce, il buffer dei delta incrementali potrebbe essere troppo corto rispetto al carico o ai tempi di riconnessione;
- se un peer passa frequentemente da `alive` a `suspected`, è plausibile che la rete simulata o l'intervallo degli heartbeat stiano introducendo transienti visibili dal failure detector;
- se i timestamp dei sensori appaiono troppo vecchi nella UI, è opportuno ridurre la pressione di pull oppure aumentare `REPLICATION_DELTA_MAXLEN`.

La dashboard è quindi non solo uno strumento dimostrativo, ma una componente importante per la diagnosi sperimentale del comportamento distribuito.

7.5 Scenari di uso consigliati

Dal punto di vista didattico e dimostrativo, gli scenari più significativi sono i seguenti:

- **convergenza in condizioni normali:** avvio del cluster e osservazione della progressiva uniformazione dello stato;
- **crash e recovery:** arresto temporaneo di un nodo e verifica del catch-up tramite delta o full sync;
- **rete degradata:** uso di ritardi e perdita pacchetti configurati nel Compose per osservare effetti su heartbeat e freschezza dei dati;
- **scalabilità osservabile:** confronto tra topologie a 2, 6 e 12 nodi tramite la stessa UI.

Per scenari di fault injection ripetuti è disponibile anche lo script `manual_tests/compose_chaos.py`, utile per introdurre arresti e riavvii controllati di singoli servizi durante l'esecuzione del cluster.

7.6 Uso di un'immagine pubblicata tramite release

Se non si desidera costruire localmente l'immagine Docker, è possibile usare direttamente una release generata dalla pipeline di continuous delivery. La release non viene però creata a ogni commit: essa viene prodotta quando a un commit viene assegnato un tag di versione, ad esempio `v1.0.0`, associato all'immagine del nodo caricata su GitHub Container Registry.

Il flusso operativo è il seguente:

1. creare localmente il tag di versione e pubblicarlo sul repository remoto;
2. aprire la sezione *Releases* del repository;
3. selezionare la release corrispondente al commit desiderato;
4. scaricare gli asset allegati, in particolare il file `.env` di esempio e il `docker-compose` minimale;
5. adattare i parametri principali del file `.env`;
6. avviare il container tramite Compose oppure usare direttamente il comando `docker run` riportato nelle note della release.

Un esempio di pubblicazione della release è:

```
git tag v1.0.0
git push origin v1.0.0
```

Un esempio tipico di esecuzione tramite gli asset allegati è:

```
cp docker-node.env.example docker-node.env
docker compose -f docker-compose.release.yml up -d
```

In alternativa, resta disponibile anche l'avvio diretto con digest:

```
docker pull ghcr.io/<owner>/<repo>@sha256:<digest>
docker run --rm -p 9000:9000 -p 10000:10000 \
  -e NODE_ID=node-1 \
  -e HOST=0.0.0.0 \
  -e PORT=9000 \
  -e WEB_API_PORT=10000 \
  -e BOOTSTRAP_PEERS= \
  ghcr.io/<owner>/<repo>@sha256:<digest>
```

Questa modalità è utile quando si vuole eseguire rapidamente un nodo coerente con uno specifico commit del branch principale, mantenendo al tempo stesso la riproducibilità dell'ambiente tramite riferimento immutabile al digest dell'immagine.

8 Self-evaluation

Questo capitolo deve essere completato individualmente da ciascun componente del gruppo prima della consegna finale. Per mantenere un tono coerente con il resto del report e, allo stesso tempo, distinguere chiaramente la responsabilità personale dei contenuti, si propone una struttura comune da replicare una volta per ogni studente.

L'obiettivo della sezione non è soltanto descrivere il contributo operativo, ma anche fornire una valutazione critica del lavoro svolto, evidenziando in modo onesto i punti di forza, i limiti emersi durante lo sviluppo e il ruolo effettivamente ricoperto nel progetto.

8.1 [Nome Cognome]

Ruolo nel progetto. Descrivere in modo sintetico e oggettivo il proprio ruolo prevalente nel gruppo. Esempi di formulazione appropriata possono riguardare: progettazione del protocollo, implementazione del runtime, sviluppo dei test automatici, gestione della containerizzazione, sviluppo della Web API, realizzazione della dashboard di osservabilità, coordinamento dell'integrazione finale o redazione del report.

Contributi principali. Riportare le attività effettivamente svolte, evitando formulazioni generiche. È preferibile descrivere responsabilità concrete, ad esempio:

- moduli progettati o implementati direttamente;
- test automatici sviluppati o mantenuti;
- scenari Docker o script di supporto realizzati;
- attività di debugging, integrazione e rifinitura;
- contributi alla documentazione tecnica e al report finale.

Punti di forza del prodotto. Indicare gli aspetti che, dal proprio punto di vista, costituiscono i risultati migliori del progetto. In un elaborato di questo tipo è opportuno concentrarsi su elementi tecnici verificabili, quali modularità dell'architettura, riproducibilità degli esperimenti, chiarezza dell'osservabilità, buona copertura di test o correttezza del comportamento di convergenza.

Debolezze e limiti. Questa parte deve esplicitare i limiti reali del sistema o del processo di sviluppo. Sono considerazioni rilevanti, ad esempio, l'assenza di autenticazione, la mancanza di persistenza durevole, la dipendenza da politiche LWW semplificate, la copertura incompleta di alcuni scenari estremi oppure la necessità di tuning manuale in topologie più dense.

Valutazione personale del contributo. Concludere con una breve riflessione critica sul proprio apporto: quali decisioni si ritengono più riuscite, quali attività hanno richiesto più iterazioni, cosa sarebbe stato utile fare meglio in termini di progettazione, testing o coordinamento.

Nota. La sottosezione precedente va duplicata per ogni membro del gruppo, sostituendo il placeholder [Nome Cognome] con i nominativi reali. Ogni studente è responsabile esclusivamente della propria parte.

9 Future works

Sebbene il progetto raggiunga gli obiettivi didattici prefissati in termini di replica distribuita, convergenza eventuale e osservabilità, l'architettura realizzata lascia spazio a diverse estensioni significative, sia dal punto di vista ingegneristico sia da quello sperimentale.

9.1 Sicurezza e hardening operativo

La prima area di evoluzione riguarda la sicurezza. Nella versione attuale il sistema opera in un contesto controllato e non implementa autenticazione dei peer, protezione dell'API HTTP né cifratura dei canali. Un'evoluzione naturale consisterebbe nell'introdurre:

- autenticazione reciproca tra nodi;
- protezione degli endpoint HTTP mediante token o mTLS;
- validazione più stringente dell'identità logica del mittente rispetto al canale di trasporto;
- policy di autorizzazione per distinguere ruoli osservativi e amministrativi.

Queste modifiche renderebbero il sistema più vicino a uno scenario reale, nel quale la tolleranza ai guasti deve convivere con requisiti minimi di trust e protezione del piano di controllo.

9.2 Persistenza e storico dei dati

Attualmente i nodi mantengono soltanto stato volatile in memoria. Questa scelta è coerente con il focus del progetto, ma limita la capacità del sistema di sopravvivere a riavvii estesi o di offrire analisi temporali di lungo periodo. Un'estensione rilevante sarebbe quindi l'introduzione di:

- persistenza locale dei record sensore e dei delta recenti;
- snapshot periodiche del metadata di membership e topology;
- meccanismi di restore all'avvio per ridurre il costo di bootstrap;
- archiviazione storica separata per analisi offline e confronto tra run sperimentali.

Questa evoluzione aprirebbe anche la possibilità di confrontare in modo più rigoroso comportamento *cold start*, tempi di recovery e perdita temporanea di informazione dopo crash estesi.

9.3 Semantiche di replica più ricche

Il criterio Last-Writer-Wins adottato è semplice, deterministico e adatto al contesto del progetto, ma costituisce anche una semplificazione. In prospettiva sarebbe interessante valutare:

- l'uso di CRDT specifici per particolari tipi di dato;
- merge policies differenziate per categoria di sensore;
- version vector o metadata causali per distinguere meglio concorrenza reale e semplice arrivo tardivo;
- strategie adattive tra replica incrementale e full sync in funzione del grado di divergenza osservato.

Queste estensioni consentirebbero di studiare in modo più approfondito il compromesso fra semplicità implementativa, quantità di metadata scambiati e qualità semantica della convergenza.

9.4 Sperimentazione su topologie più ampie

Un'altra direzione naturale riguarda l'ampliamento della scala sperimentale. Le topologie attuali permettono già di osservare fenomeni interessanti, ma un'estensione a cluster più grandi o con connettività più eterogenea renderebbe possibile:

- valutare l'impatto della densità della topologia sul traffico di gossip;
- studiare la sensibilità del failure detector in reti meno regolari;
- confrontare fan-out, intervalli di sync e dimensione dei delta buffer su carichi superiori;
- verificare la leggibilità e la sostenibilità dell'osservabilità al crescere del numero di nodi.

Da un punto di vista sperimentale, sarebbe inoltre utile affiancare metriche quantitative sistematiche su latenze di convergenza, volume di traffico e costi di recovery.

9.5 Osservabilità avanzata e strumenti di analisi

Il progetto dispone già di una buona base di introspection read-only, ma essa potrebbe essere ulteriormente estesa tramite:

- esportazione di metriche verso stack standard di monitoring;
- alerting automatico su soglie critiche di stale data o `DELTA_UNAVAILABLE`;
- tracciamento temporale persistente degli eventi di control plane;

- comparazione automatizzata tra differenti configurazioni di replica e failure detection.

In questo modo il sistema diventerebbe non solo un prototipo funzionale, ma anche una piattaforma sperimentale più matura per analizzare in maniera sistematica le conseguenze delle scelte architetturali.